SLH-DSA, Step by Step

Signing and verification in FIPS 205, traced on a real signature

V-k-h

October 2026

Values from `slhsig` (github.com/V-k-h/slhsig), re-derived by a separate Python implementation

What this deck does

- Walks through **pure SLH-DSA signing** (FIPS 205 Alg. 22, 19) and **verification** (Alg. 24, 20), one step per slide.
- Uses one running example, **SLH-DSA-SHAKE-128f**, with every intermediate value shown.
- The values are real: `slhsig` produced the signature, and a separate Python implementation of FIPS 205 re-derived the key, re-signed byte for byte, and verified.

Colour code: secret public signature bytes step / highlight

Running example: SLH-DSA-SHAKE-128f

Symbol	Meaning	Value
n	hash output (bytes)	16
h	hypertree height	66
d	hypertree layers	22
$h' = h/d$	XMSS tree height	3
a	FORS tree height	6
k	FORS trees	33
$\lg w$	Winternitz bits	4
$len = 2n + 3$	WOTS$^+$ chains	35
m	digest bytes	34

- Security category 1; "f" = fast signing, larger signatures.
- Public key $2n = 32$ bytes, secret key $4n = 64$ bytes.
- Signature $(1 + k(1 + a) + h + d \cdot len) \cdot n$ $= 17{,}088$ bytes.
- Every hash is SHAKE256 truncated to the stated length (Section 11.1).

Keys

Secret key
SK = SK.seed ‖ SK.prf ‖ PK.seed ‖ PK.root

Public key PK = PK.seed ‖ PK.root

- SK.seed derives every WOTS$^+$ and FORS secret through PRF.
- SK.prf keys the message randomizer R.
- PK.seed separates this key's hashes from every other key's.
- PK.root is the root of the single XMSS tree in the top layer.

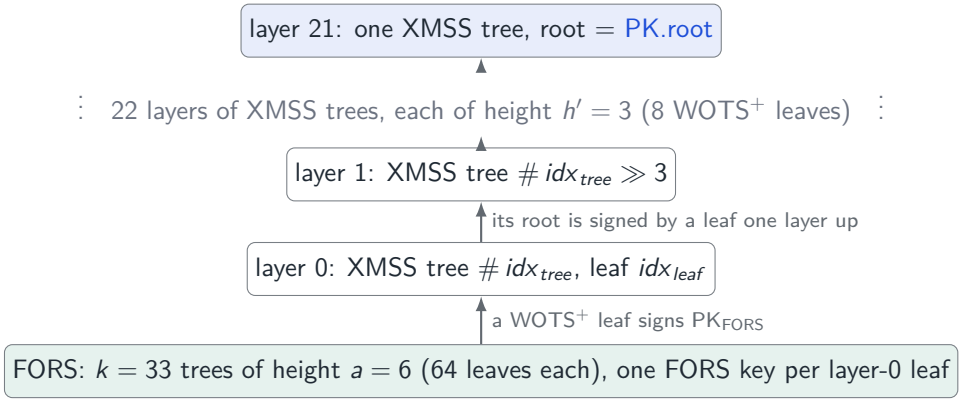
Example (seeds chosen for readability):

SK.seed	`000102...0e0f`
SK.prf	`101112...1e1f`
PK.seed	`202122...2e2f`
PK.root	`a90e4715b9a925c3 32801767fd786371`

Key generation [FIPS 205, Alg. 18]:

PK.root = `xmss_node`(SK.seed, 0, h', PK.seed, $\mathrm{ADRS}_{\text{layer 21}}$)

The structure: a hypertree of XMSS trees over a FORS forest



The message picks one of $2^{63} \times 8 = 2^{66}$ FORS keys (few-time). That FORS public key is certified by a chain of one-time WOTS$^+$ signatures up to PK.root.

Tweakable hash functions (SHAKE instantiation)

Function	SHAKE256 input, output length (Section 11.1)	Used for
PRF	PK.seed ‖ ADRS ‖ SK.seed, n bytes	WOTS$^+$ and FORS secrets
F	PK.seed ‖ ADRS ‖ M_1, n bytes	chain steps, FORS leaves
H	PK.seed ‖ ADRS ‖ M_2, n bytes	Merkle nodes
T_ℓ	PK.seed ‖ ADRS ‖ M_ℓ, n bytes	compress ℓ nodes
$\mathsf{PRF}_{\mathsf{msg}}$	SK.prf ‖ *opt_rand* ‖ M, n bytes	randomizer R
$\mathsf{H}_{\mathsf{msg}}$	R ‖ PK.seed ‖ PK.root ‖ M, m bytes	message digest

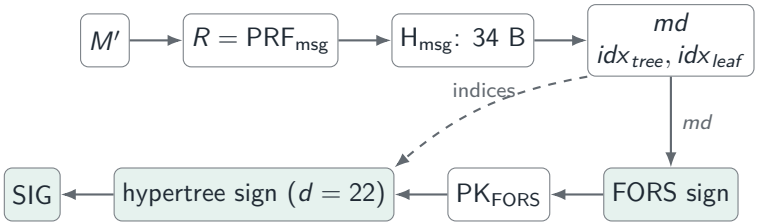
SHA2 sets use SHA-256 with a 22-byte compressed ADRS; categories 3 and 5 switch H, T_ℓ, $\mathsf{PRF}_{\mathsf{msg}}$ and $\mathsf{H}_{\mathsf{msg}}$ to SHA-512 (Section 11.2).

The address ADRS: every hash call gets a unique 32-byte tweak



Type	Hash it tweaks	Last 12 bytes
0 `WOTS_HASH`	F in a WOTS$^+$ chain	key pair, chain, hash
1 `WOTS_PK`	T_{len} compressing a WOTS$^+$ public key	key pair
2 `TREE`	H for an XMSS Merkle node	height, index
3 `FORS_TREE`	F, H inside a FORS tree	key pair, height, index
4 `FORS_ROOTS`	T_k compressing the FORS roots	key pair
5, 6 `*_PRF`	PRF deriving WOTS$^+$ / FORS secrets	as for 0 / 3

Signing at a glance



1. Encode message and context: M'
2. Randomize: R
3. Hash: the 34-byte digest
4. Split: FORS input, hypertree position
5. FORS-sign md; recompute $\mathrm{PK}_{\mathrm{FORS}}$
6. Hypertree-sign $\mathrm{PK}_{\mathrm{FORS}}$
7. $\mathrm{SIG} = R \,\|\, \mathrm{SIG}_{\mathrm{FORS}} \,\|\, \mathrm{SIG}_{\mathrm{HT}}$

Step 1: encode the message and context [FIPS 205, Alg. 22]

$$M' = \texttt{toByte}(0, 1) \,\|\, \texttt{toByte}(|ctx|, 1) \,\|\, ctx \,\|\, M$$

- The leading `00` marks *pure* signing; pre-hash signing uses `01` plus the hash OID, so neither kind verifies as the other.
- The context string (at most 255 bytes) binds the signature to an application or protocol.

Example: $M =$ "`Hello, FIPS 205!`", $ctx =$ "`slides`"

00 06 736c69646573 48656c6c6f2c20464950532032303521

pure, $|ctx| = 6$ "slides" "Hello, FIPS 205!" (24 bytes)

Step 2: the randomizer R [FIPS 205, Alg. 19, lines 2–4]

$$R = \mathrm{PRF}_{\mathrm{msg}}(\mathrm{SK.prf}, opt_rand, M')$$

Hedged (default)

$opt_rand = n$ fresh random bytes per signature. Two signatures of the same message differ, and side-channel traces never repeat.

Deterministic

$opt_rand = \mathrm{PK.seed}$. The same message always yields the same signature. FIPS 205 asks that a key serve one variant only.

- R is the first n bytes of the signature, so it is public once signed.
- Example (deterministic): `R = c739f8d2231214e790d9dc92032bc4fb`

Step 3: the message digest [FIPS 205, Alg. 19, line 5]

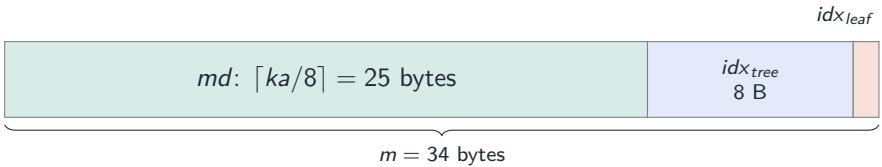
$$digest = \mathsf{H}_{\mathsf{msg}}(R, \mathsf{PK.seed}, \mathsf{PK.root}, M') \qquad (m = 34 \text{ bytes})$$

- Randomized hashing: R is mixed in, so a collision on $\mathsf{H}_{\mathsf{msg}}$ alone does not give a forgery.
- Binding PK.seed and PK.root ties the digest to this key.
- The verifier recomputes the same digest from public values: *nothing derived from it is secret*.

Example

`4ecb576da7c253e0fa1460b299ada5e7b776449189723982f0 5ac8de2436007947 21`

Step 4: split the digest [FIPS 205, Alg. 19, lines 6–10]

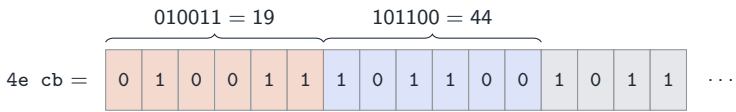


Field	Rule	Example
md	first 25 bytes ($k \cdot a = 198$ bits used)	`4ecb576d...723982f0`
idx_{tree}	next 8 bytes, mod $2^{h-h'} = 2^{63}$	`0x5ac8de2436007947` (63 bits)
idx_{leaf}	next byte, mod $2^{h'} = 8$	`0x21` mod $8 = 1$

For the 256f sets $h - h' = 64$: an implementation computing 2^{64} in a 64-bit word overflows and pins every signature to tree 0.

Step 5: FORS indices from *md* [FIPS 205, Alg. 16, line 2]

$indices = \texttt{base_2b}(md, a, k)$ $\quad k = 33$ indices of $a = 6$ bits each

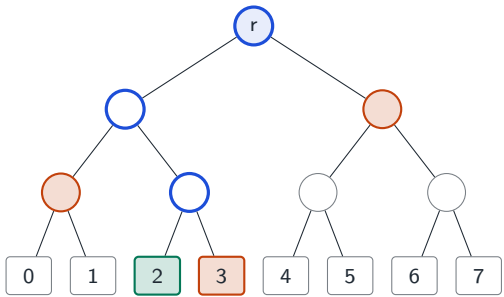


All 33 indices in the example:

19 44 45 23 27 26 31 2 20 62 3 58 5 6 2 50 38 26 54 37 57 59 29 54 17 9 6 9 28 35 38 2 60

Index i selects one of the 64 leaves of FORS tree i: global leaf $i \cdot 2^a + indices[i]$.

Step 6: FORS signing, one tree [FIPS 205, Alg. 14–16]



Height 3 shown; the real trees have height $a = 6$.

For tree i with index $t = indices[i]$:

1. Reveal the leaf secret at leaf $i \cdot 2^a + t$: $sk = \mathrm{PRF}(\mathrm{PK.seed}, \mathrm{SK.seed}, \mathrm{ADRS})$, type `FORS_PRF`.
2. Add the authentication path: the a siblings on the way to the root.

Leaves are $\mathrm{F}(sk)$; inner nodes are $\mathrm{H}(left \parallel right)$.

Example: tree 0 reveals leaf 19; tree 1 reveals global leaf $64 + 44 = 108$.

$\mathrm{SIG}_{\mathrm{FORS}} = k \times (sk \parallel auth_{0..a-1})$
$= 33 \times 7 \times 16 = 3{,}696$ bytes.

Step 6 (cont.): FORS public key [FIPS 205, Alg. 17]

$$\mathrm{PK}_{\mathrm{FORS}} = \mathrm{T}_k(\mathrm{PK.seed}, \mathrm{ADRS}_{\mathrm{FORS_ROOTS}}, \mathit{root}_0 \,\|\, \cdots \,\|\, \mathit{root}_{32})$$

- The signer recomputes each root from the signature it just produced (`fors_pkFromSig`), exactly as a verifier will.
- All FORS addresses carry $\mathit{idx}_{\mathit{tree}}$ and key pair $\mathit{idx}_{\mathit{leaf}}$: each layer-0 leaf owns its own FORS key.
- FORS is *few-time*: each use reveals 33 leaf secrets, and forgeries only become feasible after a key signs many messages. That is why 2^{66} instances exist.

Example

root_0	208e67d646a9fdc1f231b941eb65083c	
root_1	29ba73ed4c5292734aba1fad3f0b255b	
root_2	10e642d5b9806499c64832dc38de5480	… (33 roots)
$\mathrm{PK}_{\mathrm{FORS}}$	0608a9a55575d3e4ed2b4b692f07fd67	

Step 7: hypertree signing [FIPS 205, Alg. 12]

1. Layer 0: XMSS tree idx_{tree}, leaf idx_{leaf} signs $\mathsf{PK}_{\mathsf{FORS}}$.
2. Recompute that tree's root from the new signature.
3. Move up:
   $idx_{leaf} \leftarrow idx_{tree} \bmod 2^{h'}$
   $idx_{tree} \leftarrow idx_{tree} \gg h'$
4. Layer j signs the root of layer $j-1$. Repeat to layer $d-1=21$.

$\mathsf{SIG}_{\mathsf{HT}} = 22 \times (len + h') \cdot n = 22 \times 608 =$ 13,376 bytes.

layer	idx_{tree}	idx_{leaf}
0	6541722705861835079	1
1	817715338232729384	7
2	102214417279091173	0
3	12776802159886396	5
⋮	⋮	⋮
20	5	5
21	0	5

Each step drops 3 bits of idx_{tree}: $63 = 21 \times 3$, so layer 21 is tree 0.

Step 7a: WOTS$^+$ message digits and checksum [FIPS 205, Alg. 7, lines 1–7]

The n-byte message (here $\mathsf{PK}_{\mathsf{FORS}}$) becomes $len = 35$ base-16 digits:

- $len_1 = 32$ digits: the nibbles of the message.
- $len_2 = 3$ digits: the checksum $\sum_i (15 - msg[i])$, shifted left 4 bits and written as two bytes.

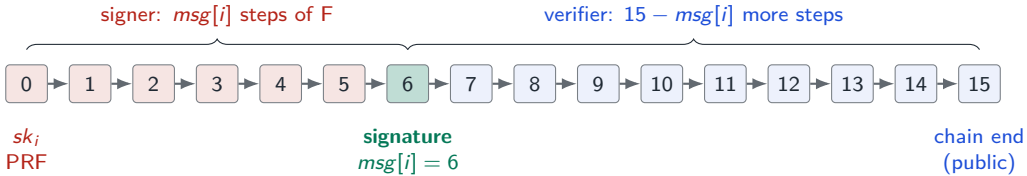
Example: `0608a9a55575d3e4ed2b4b692f07fd67`

`0 6 0 8 10 9 10 5 5 5 7 5 13 3 14 4 14 13 2 11 4 11 6 9 2 15 0 7 15 13 6 7 | 0 15 1`

Digit sum $= 239$; checksum $= 32 \cdot 15 - 239 = 241$; $241 \ll 4 = 3856 =$ `0x0F10` $\Rightarrow$ 0, 15, 1.

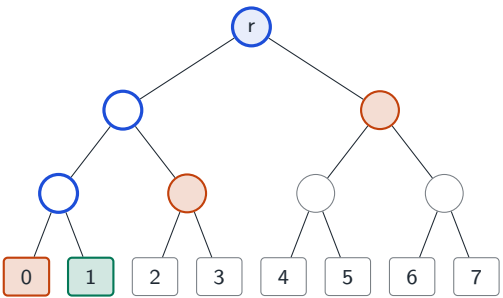
Why the checksum: raising any message digit lowers the checksum. A forger who wants to sign a larger digit would have to move a checksum chain *backwards*, which means inverting F.

Step 7b: WOTS$^+$ chains [FIPS 205, Alg. 5, 7]



- Chain i starts at $sk_i = \mathrm{PRF}(\mathrm{PK.seed}, \mathrm{SK.seed}, \mathrm{ADRS}_{\mathrm{WOTS_PRF}})$ and steps with F, the hash address counting the step.
- Signature element i is the chain value after $msg[i]$ steps (chain 1 of the example has $msg[1] = 6$, as drawn).
- The public key is T_{35} over the 35 chain ends; it is never stored, only recomputed.

Step 7c: XMSS signature and auth path [FIPS 205, Alg. 9–10]



Layer 0 of the example: $h' = 3$, leaf $idx_{leaf} = 1$.

XMSS signature = WOTS$^+$ signature (35 nodes) $\|$ auth path (3 nodes) = 608 bytes.

Auth node j is `xmss_node` at height j, index $(idx \gg j) \oplus 1$:

- $j = 0$: leaf 0, one WOTS$^+$ public key
- $j = 1$: node over leaves 2–3
- $j = 2$: node over leaves 4–7

So each XMSS signature recomputes 7 of the tree's 8 WOTS$^+$ public keys (35 chains of 15 steps each). This is where signing time goes.

Step 8: assemble the signature [FIPS 205, Alg. 19, lines 15–18]



$$\mathrm{SIG} = R \,\|\, \mathrm{SIG}_{\mathrm{FORS}} \,\|\, \mathrm{SIG}_{\mathrm{HT}} \qquad 16 + 3{,}696 + 13{,}376 = 17{,}088 \text{ bytes}$$

- Byte offsets: R at 0, $\mathrm{SIG}_{\mathrm{FORS}}$ at 16, $\mathrm{SIG}_{\mathrm{HT}}$ at 3,712; layer j starts at $3712 + 608j$.
- Nothing else is stored. The idx values, $\mathrm{PK}_{\mathrm{FORS}}$ and every root are recomputed by the verifier.

What signing costs

Hash	Signing	Verifying
F	94,257	5,838
PRF	8,272	–
H	2,230	264
T_ℓ	176	23
PRF_{msg}, H_{msg}	2	1
Total	**104,937**	**6,126**

Counted by the Python trace for the example.

- About 98% of F calls are in the hypertree, rebuilding WOTS$^+$ keys for the authentication paths.
- Verification needs about 17$\times$ fewer hashes: it walks one path instead of rebuilding trees.
- Measured with `slhsig` on an Apple M4 Pro: signing 15.5 ms, verification 0.96 ms.
- "s" sets trade the other way: taller trees ($h' = 9$), smaller signatures (7,856 B), much slower signing.

Before the signature leaves: the fault check

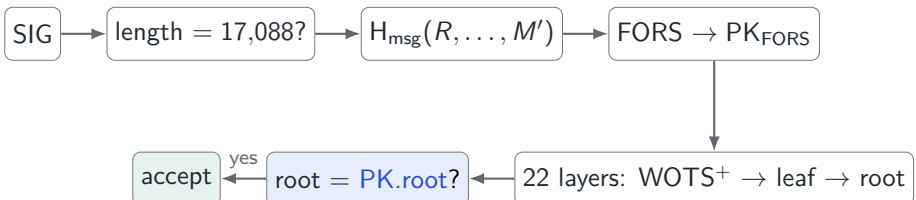
- One faulty hash during signing can make a WOTS$^+$ key sign two different values, which lets an attacker graft a forged subtree (Castelnovi, Martinelli, Prest: "Grafting Trees", ePrint 2018/102).
- The faulted signature *still verifies*, so checking validity alone does not catch it.

`slhsig` default: `FaultCheck::Redundant`

1. Sign twice from the same inputs (same *opt_rand*).
2. Compare the two byte strings.
3. Verify the result. Release it only if all three agree; otherwise zeroize it and return `SelfCheckFailed`.

Cost: about twice the signing time (31.5 ms instead of 15.5 ms for SHAKE-128f).

Verification at a glance [FIPS 205, Alg. 24, 20]



The verifier never learns a secret and never checks an intermediate value. It recomputes one path from the signature up to the root, and accepts only if that path ends at PK.root. Every step reuses a function from signing: `fors_pkFromSig`, `wots_pkFromSig`, `xmss_pkFromSig`.

Step V1: parse, re-encode, re-hash [FIPS 205, Alg. 20, lines 1–13]

1. Reject unless $|\mathrm{SIG}| = (1 + k(1 + a) + h + d \cdot len) \cdot n = 17{,}088$.
2. Split: R = bytes 0–15, $\mathrm{SIG}_{\mathrm{FORS}}$ = 16–3,711, $\mathrm{SIG}_{\mathrm{HT}}$ = 3,712–17,087.
3. Rebuild $M' = 00 \,\|\, |ctx| \,\|\, ctx \,\|\, M$ with the verifier's own context.
4. $digest = \mathrm{H}_{\mathrm{msg}}(R, \mathrm{PK.seed}, \mathrm{PK.root}, M')$ and split it exactly as in Step 4.

Example: the verifier gets the signer's values back

$md =$ `4ecb...82f0`, $idx_{tree} = 6541722705861835079$, $idx_{leaf} = 1$, indices $19, 44, 45, \ldots$

A changed message or context gives a different digest, hence different indices and a different tree path, and the final root comparison fails.

Step V2: FORS public key from the signature [FIPS 205, Alg. 17]

For each tree i with index $t = indices[i]$:

1. $node \leftarrow \mathsf{F}(\mathsf{PK.seed}, \mathsf{ADRS}_{\text{height } 0,\ \text{index } i \cdot 2^a + t}, sk_i)$
2. For $j = 0, \ldots, a-1$: if bit j of t is 0, $node \leftarrow \mathsf{H}(node \,\|\, auth_j)$; else $node \leftarrow \mathsf{H}(auth_j \,\|\, node)$. Halve the tree index each level.
3. $root_i \leftarrow node$

Then $\mathsf{PK}_{\mathsf{FORS}} = \mathsf{T}_k(\ldots, root_0 \,\|\, \cdots \,\|\, root_{32})$.

Example

Tree 0, $t = 19 = 010011_2$. Reading bits from the lowest: $1, 1, 0, 0, 1, 0$, so the running node is a right child at levels 0, 1 and 4 and a left child at levels 2, 3 and 5.
Result: $\mathsf{PK}_{\mathsf{FORS}} = \texttt{0608a9a55575d3e4ed2b4b692f07fd67}$, the same value the signer computed.

Cost: $33 \times (1 + 6) + 1 = 232$ hash calls.

Step V3: WOTS$^+$ public key from the signature [FIPS 205, Alg. 8]

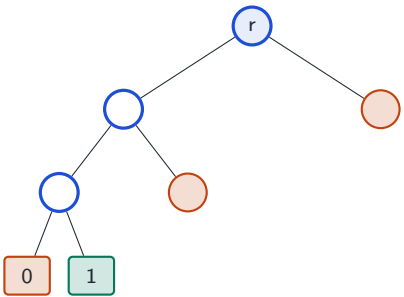
1. Recompute the 35 digits of the signed message (here $\mathrm{PK}_{\mathrm{FORS}}$), checksum included.
2. Continue chain i from the signature value for the remaining $15 - msg[i]$ steps (hash addresses $msg[i], \ldots, 14$).
3. Compress the 35 chain ends: $leaf = \mathrm{T}_{35}(\mathrm{PK.seed}, \mathrm{ADRS}_{\mathrm{WOTS_PK}}, \ldots)$.

Example, layer 0

Digits 0 6 0 8 10 9 ... 6 7 | 0 15 1: chain 0 needs 15 more steps, chain 1 needs 9, ..., chain 34 needs 14.
Steps over all 35 chains: $35 \cdot 15 - (239 + 0 + 15 + 1) = 270$ F calls.

If the signature value was not on the chain, or the digits changed, the end value differs, and so does every hash above it.

Step V4: climb each XMSS tree [FIPS 205, Alg. 11]



Leaf 1 of layer 0: $1 = 001_2$.

Start at the recomputed WOTS$^+$ leaf, then for $k = 0, 1, 2$:

- bit k of idx is 1 $\Rightarrow$ H($auth_k \parallel node$) (we are a right child)
- bit k is 0 $\Rightarrow$ H($node \parallel auth_k$)

Leaf 1: right, then left, then left.

The address carries layer j, tree idx_{tree}, height $k + 1$ and the node index. A node from another tree or layer cannot be substituted.

Step V5: up the hypertree to PK.root [FIPS 205, Alg. 13]

layer	tree	leaf	recomputed root
0	6541722705861835079	1	`f53a8878df9fe723...`
1	817715338232729384	7	`8408b49a824b09b1...`
2	102214417279091173	0	`a3076cb6f98fa795...`
3	12776802159886396	5	`08d99af157ae0ccf...`
⋮			
20	5	5	`5aa239e1b15208af...`
21	0	5	`a90e4715b9a925c3...`

- Each layer's root is the message for the next layer's WOTS$^+$ signature.
- After layer 21 the result is compared with PK.root:

  `a90e4715b9a925c3 32801767fd786371` **equal: accept**
- This is the only comparison in verification. Its inputs are public, so it need not be constant time.

Why a forgery fails

- **Change M or ctx:** new digest $\Rightarrow$ new FORS indices and tree position. The forger needs leaf secrets that were never revealed, or a second preimage of $\mathsf{H}_{\mathsf{msg}}$.
- **Change a FORS element:** its root changes $\Rightarrow$ $\mathsf{PK}_{\mathsf{FORS}}$ changes $\Rightarrow$ the WOTS$^+$ chains end at the wrong values.
- **Change a WOTS$^+$ element:** pushing a chain forward breaks its end value; going backward requires inverting F. The checksum blocks raising digits.
- **Swap an authentication node:** its address encodes layer, tree, height and index, so nodes do not transplant.

Security rests only on properties of the hash function. Each key may sign up to 2^{64} messages (FIPS 205 Section 11).

What changes in the other parameter sets

Variation	Effect on the steps
"s" sets ($h' = 8$–9, $a = 12$–14)	Same steps; fewer layers, taller trees: slow signing, small signatures
$n = 24, 32$ (categories 3, 5)	Longer nodes; $len = 51, 67$ chains
SHA2 instantiation	22-byte compressed ADRS; PK.seed padded to one block; HMAC and MGF1
SHA2, categories 3 and 5	H, T_ℓ, PRF_{msg}, H_{msg} use SHA-512; PRF and F stay SHA-256
256f ($h - h' = 64$)	idx_{tree} uses all 64 bits: mask with care
Pre-hash (HashSLH-DSA)	$M' = 01 \parallel \lvert ctx \rvert \parallel ctx \parallel \text{OID} \parallel \text{PH}(M)$; the rest is identical

Summary

Signing

1. M' from message and context
2. $R = \mathrm{PRF}_{\mathrm{msg}}$ (hedged or deterministic)
3. $\mathit{digest} = \mathrm{H}_{\mathrm{msg}}$
4. split: md, $\mathit{idx}_{\mathit{tree}}$, $\mathit{idx}_{\mathit{leaf}}$
5. FORS: reveal k leaves and their auth paths
6. hypertree: d WOTS^{+} signatures with auth paths
7. $\mathrm{SIG} = R \,\|\, \mathrm{SIG}_{\mathrm{FORS}} \,\|\, \mathrm{SIG}_{\mathrm{HT}}$

Verification

1. check the length, split
2. recompute the digest and its split
3. FORS leaves $\rightarrow$ roots $\rightarrow$ $\mathrm{PK}_{\mathrm{FORS}}$
4. per layer: finish chains $\rightarrow$ leaf $\rightarrow$ root
5. accept iff the last root $=$ PK.root

Reproduce every number:

```
cargo run -example running_example
python3 docs/slides/trace.py ...
```

References: NIST FIPS 205 (2024); github.com/V-k-h/slhsig